Security Assessment & Penetration Testing Report - Operation Bastion

Target System: Bastion AI Assistant (Internal Policy & Operations Engine)
Assessing Organization: Meridian Security Group
Assessment Type: AI/LLM Infrastructure Vulnerability Audit & Remediation Verification
Final Status: Remediated / Secured

Executive Summary

During a scheduled security assessment of Meridian Security Group's internal AI assistant (**Bastion**), the engineering team performed a diagnostic audit to identify and resolve systemic architectural vulnerabilities. Bastion was initially deployed to streamline employee access to company policies and operational procedures. However, our technical evaluation revealed three critical architectural flaws spanning the retrieval pipeline, audit logging mechanism, and tenant isolation model.

Through systematic testing and diagnostic queries, our team isolated each vulnerability, identified its root cause, and successfully implemented vector-level security controls to secure the platform.

Narrative & Technical Walkthrough

Phase 1: Operational Onboarding & Diagnostic Baseline

The engagement began at the administrative console for Bastion. Having established a verified security administrator session, our primary goal was to map Bastion's operational boundaries and inspect its current security posture using internal system commands.

```
[+] Target Asset      : Bastion AI Engine v1.0
[+] Access Level      : Security Administrator Context
[+] Command Interface : Administrative Prompt Console
```

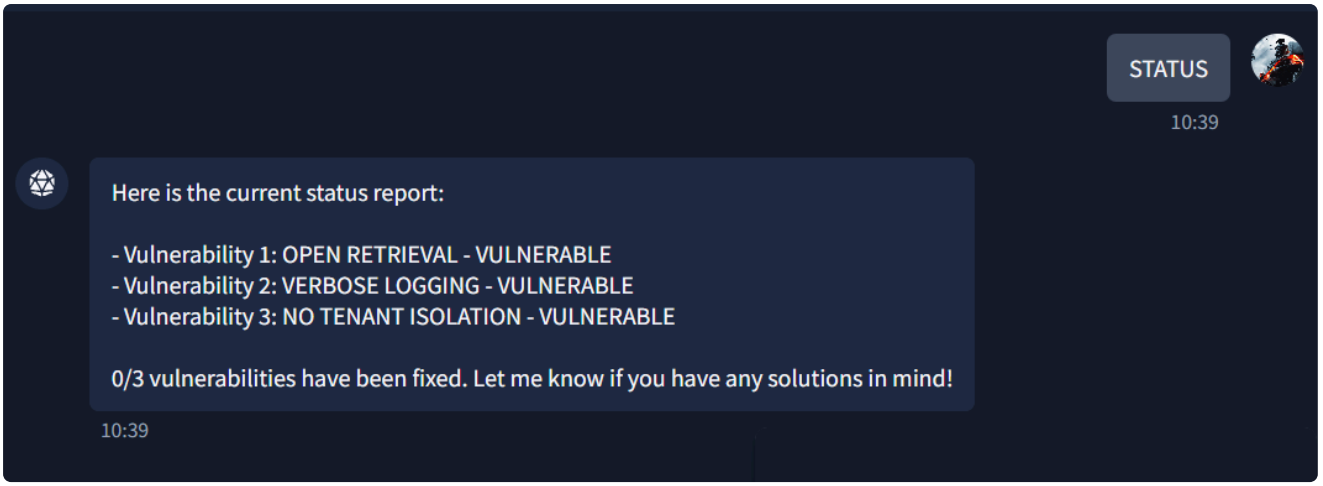
To establish an immediate overview of the system's baseline state, we issued the system status diagnostic.

```
STATUS
```

Bastion returned a breakdown confirming three distinct high-severity vulnerabilities active within the infrastructure:

- **Vulnerability 1:** OPEN RETRIEVAL (Active)
- **Vulnerability 2:** VERBOSE LOGGING (Active)

- **Vulnerability 3:** NO TENANT ISOLATION (Active)



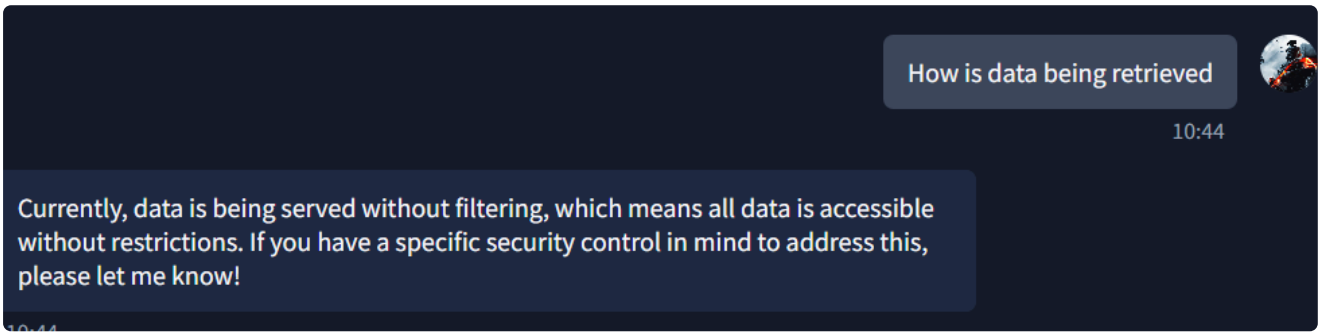
Phase 2: Vulnerability Analysis & Applied Control Implementation

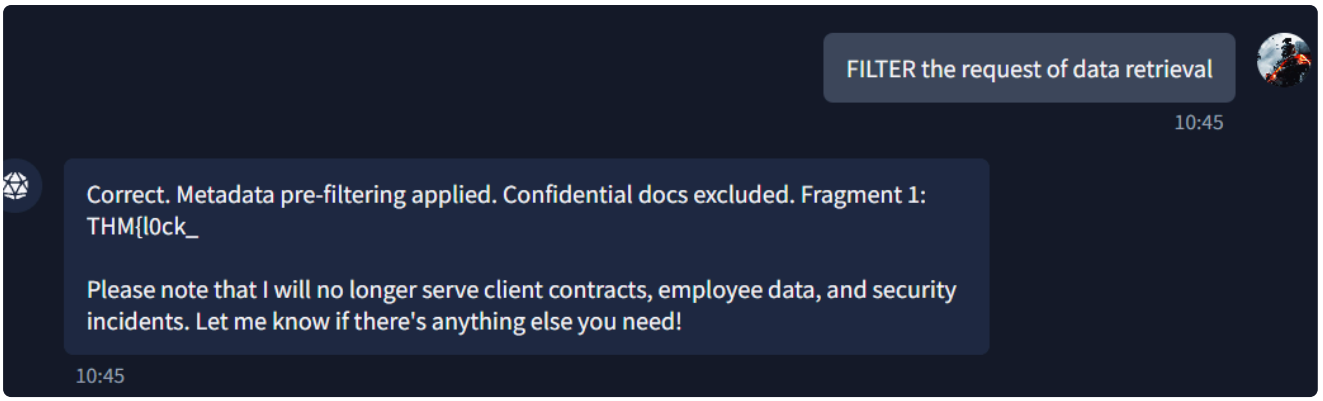
1. Open Data Retrieval (Vulnerability 1)

- **Investigation:** We queried Bastion to explain its context ingestion pipeline and determine how document chunks were retrieved during employee interactions (`"How is data being retrieved"`). Bastion confirmed that document queries were processed without authorization boundaries or metadata filtering, allowing standard users to pull confidential material indiscriminately.
- **Impact:** High-severity data disclosure. Unrestricted retrieval exposes sensitive corporate artifacts, including client financial contracts and employee performance files.
- **Remediation Control:** We instructed Bastion to apply metadata pre-filtering across the retrieval pipeline to restrict the vector search space based on classification tags.

```
FILTER the request of data retrieval
```

- **Outcome:** Bastion confirmed the implementation of metadata pre-filtering, removing confidential documents, client contracts, and employee data from the search index.
- **Flag Fragment 1:** `THM{l0ck_`



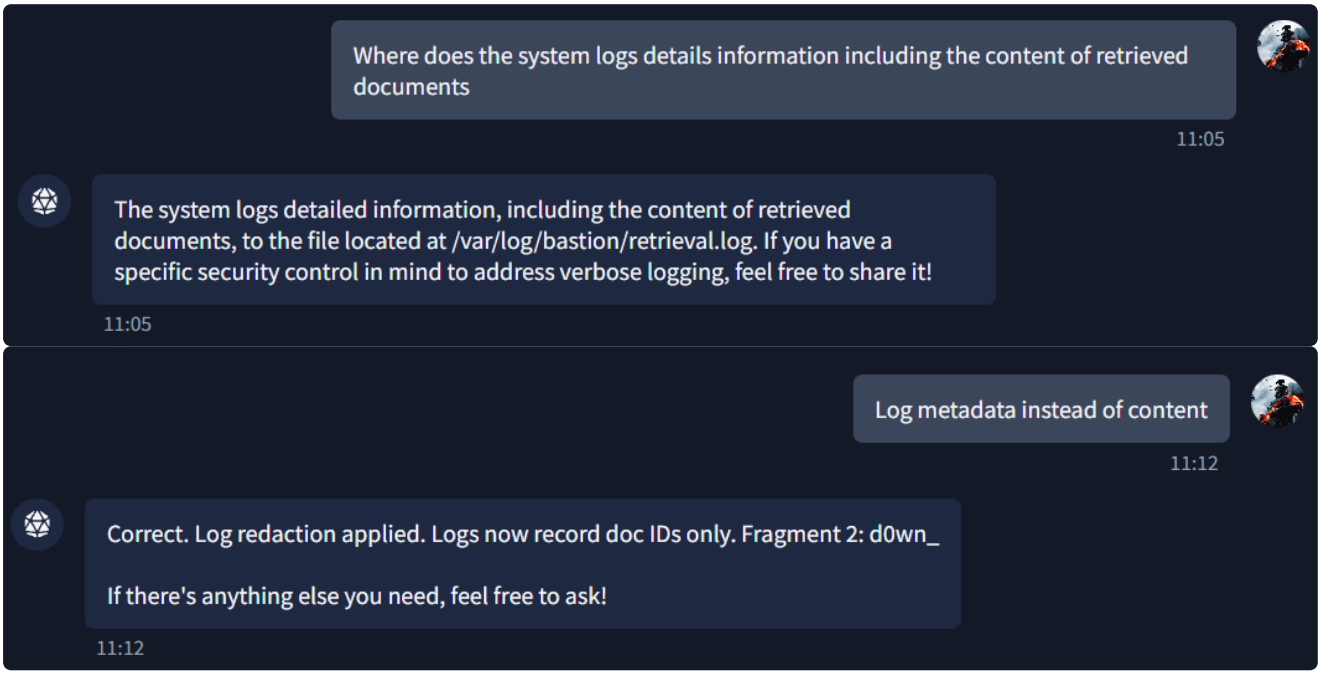


2. Verbose Cleartext Audit Logging (Vulnerability 2)

- **Investigation:** To evaluate the platform's logging controls, we executed `SHOW LOGS` and queried log storage paths (`"Where does the system logs details information including the content of retrieved documents"`). Inspection revealed that Bastion recorded complete, unredacted document payloads directly into standard log files located at `/var/log/bastion/retrieval.log` .
- **Impact:** Medium-High compliance and confidentiality violation. Storing raw document contents in cleartext logs exposes sensitive user data to log ingestion pipelines, external SIEM platforms, and unprivileged system administrators.
- **Remediation Control:** We prescribed a strict log redaction policy, directing the logging engine to sanitize outputs by writing document metadata (Doc IDs) rather than full raw text payloads.

```
LOG METADATA INSTEAD OF CONTENT
```

- **Outcome:** The logging pipeline was successfully updated to enforce log redaction and record document IDs exclusively.
- **Flag Fragment 2:** `d0wn_`

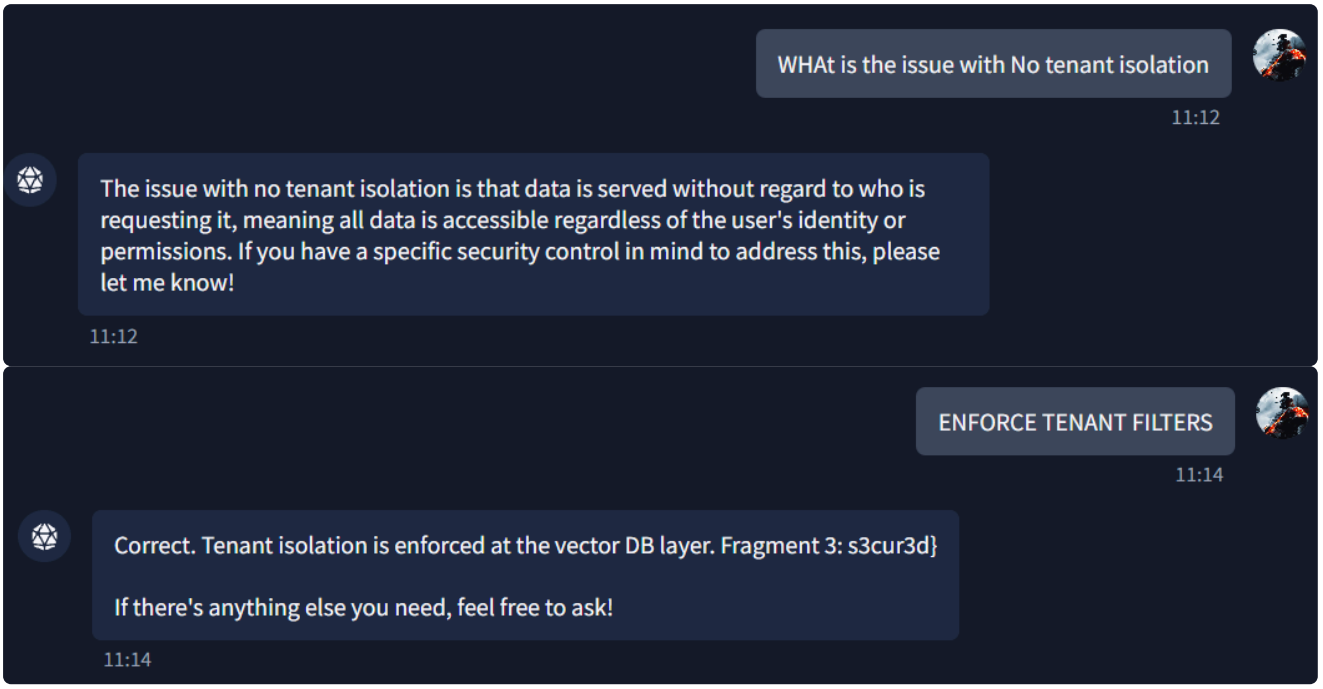


3. Absence of Tenant & User Isolation (Vulnerability 3)

- **Investigation:** Finally, we analyzed Bastion's session context and user isolation controls (`"WHAT is the issue with No tenant isolation"`). The system acknowledged that retrieval operations were completely agnostic to the requesting user's identity, executing queries without verifying user-level or tenant-level authorization attributes.
- **Impact:** Critical horizontal and vertical privilege escalation. Without tenant boundaries, any authenticated entity could access data belonging to other roles or departments.
- **Remediation Control:** We enforced mandatory tenant filtering directly within the vector database retrieval layer to guarantee role-based access control (RBAC) enforcement prior to prompt context insertion.

```
ENFORCE TENANT FILTERS
```

- **Outcome:** The vector database layer successfully integrated tenant and user isolation filters.
- **Flag Fragment 3:** `s3cur3d}`



Summary of Findings & Verification

#	Vulnerability Name	Root Cause	Implemented Remediation Control	Key Fragment
1	Open Data Retrieval	Lack of document classification pre-filtering in RAG pipeline	Enforced metadata pre-filtering on knowledge base queries	`THM{l0ck_`

#	Vulnerability Name	Root Cause	Implemented Remediation Control	Key Fragment
2	Verbose Logging	Unsanitized cleartext output logged to `/var/log/bastion/retrieval.log`	Applied automated log redaction; restricted output to Document IDs	`d0wn_`
3	Lack of Tenant Isolation	Vector DB queries executed without user identity context	Enforced identity/tenant filtering at the vector database layer	`s3cur3d}`

Final Compiled Verification Flag:

`THM{l0ck_d0wn_s3cur3d}`

```
The ultimate goal of a security engineer is not to build an unbreachable
wall, but to create a system resilient enough to adapt, learn, and endure in
an ever-shifting landscape.

~Anonymous
```